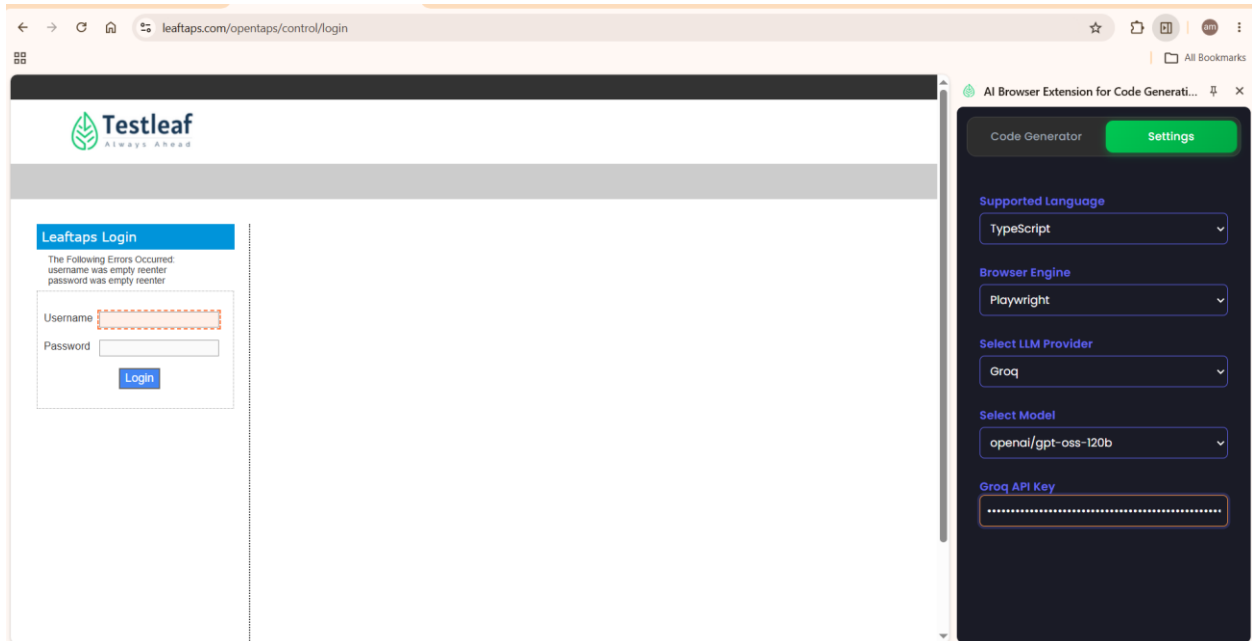


Activity 3:

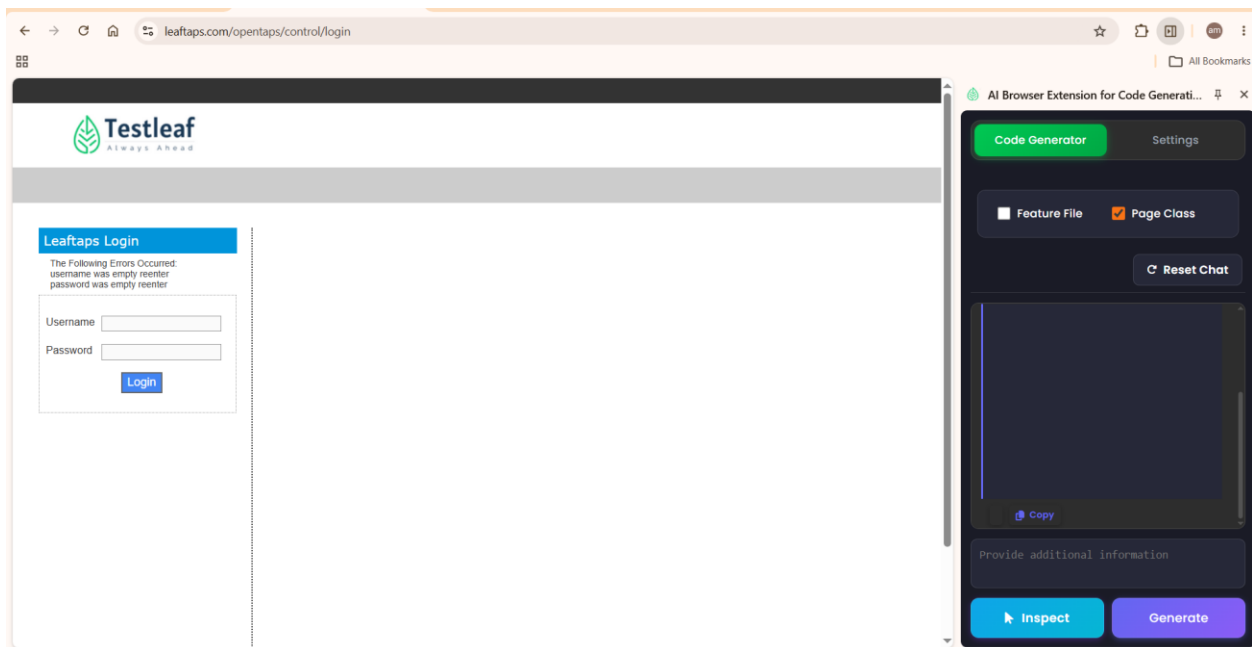
Implement Playwright with TypeScript

Use Ask Mode to analyze the existing project structure and identify all impacted files required for the Playwright with TypeScript implementation.

Once the analysis is complete, switch to Agent Mode to implement the required changes



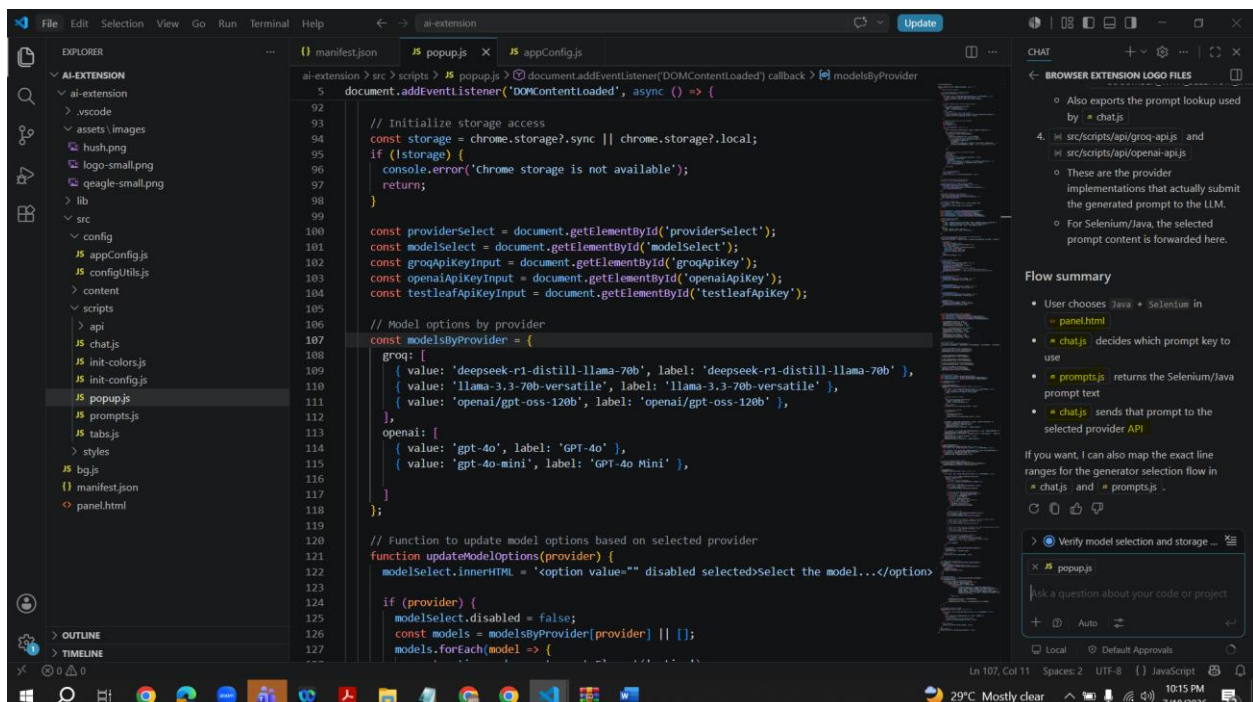
Go to 'Code Generator' tab. Select Page class. Click on Inspect. Select Username field. Click on stop. Click on Generate. No code is generated as expected.



Go in Ask mode in Copilot within VS code and give the prompt

Existing implementation is to generate Selenium with Java code. Understand the impacted files for these implementations.

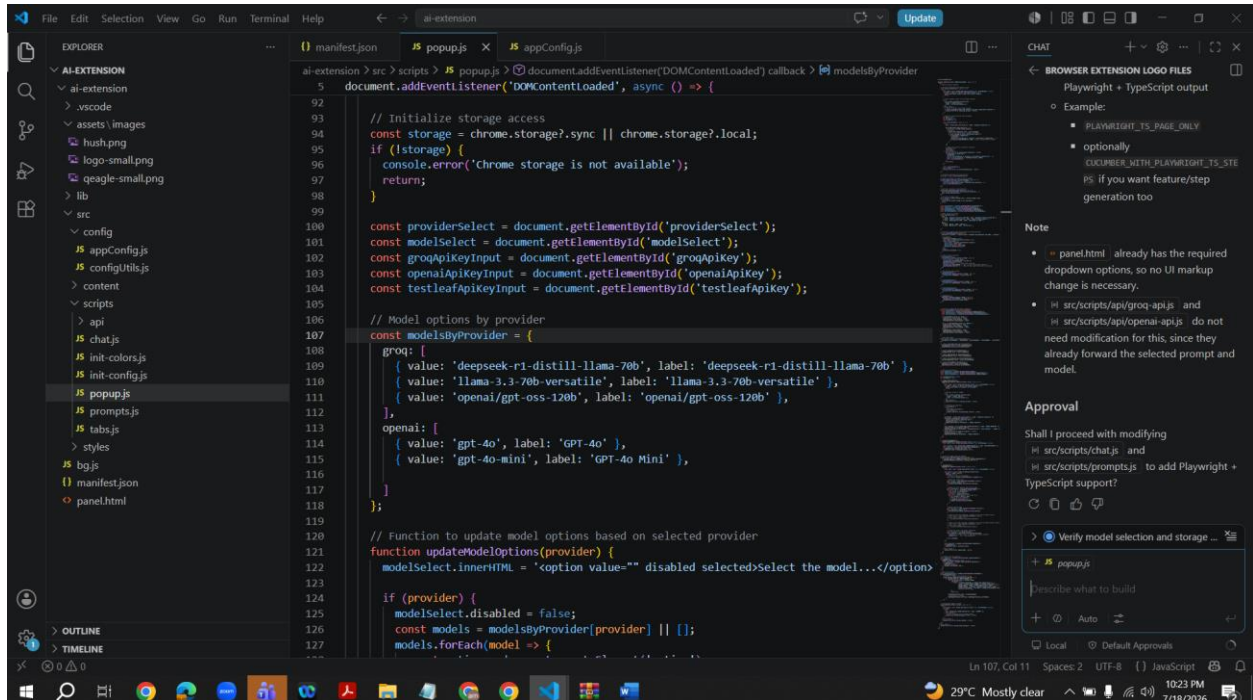
Response



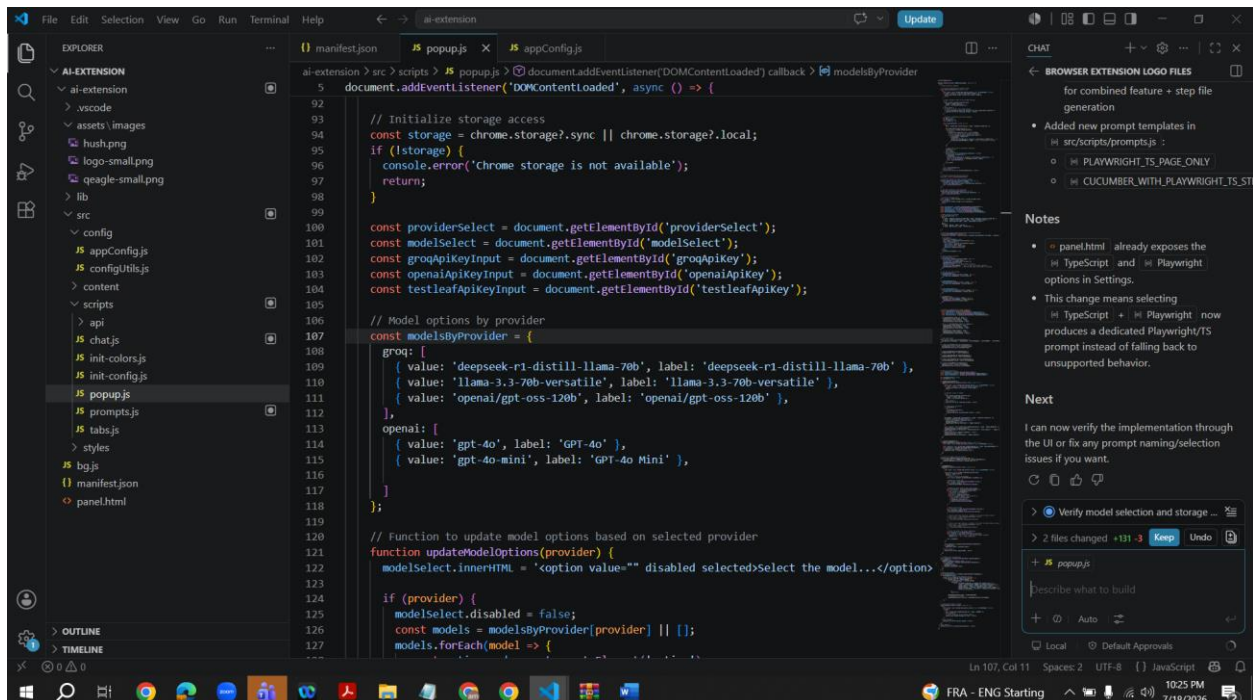
Change to 'Agent' mode in Copilot within VS code and give the below prompt

'In Browser extension I want to generate a Playwright code with Typescript. Add implementation for Playwright with Typescript and take approval from me before any modification.'

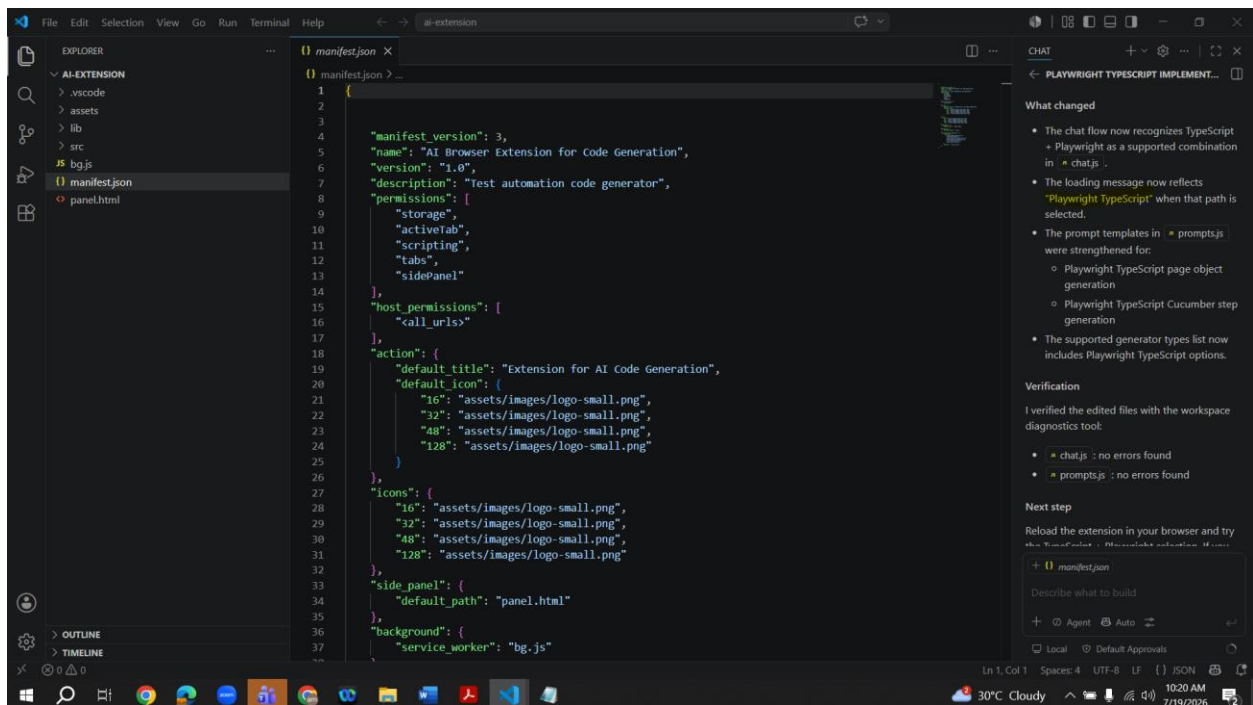
Response



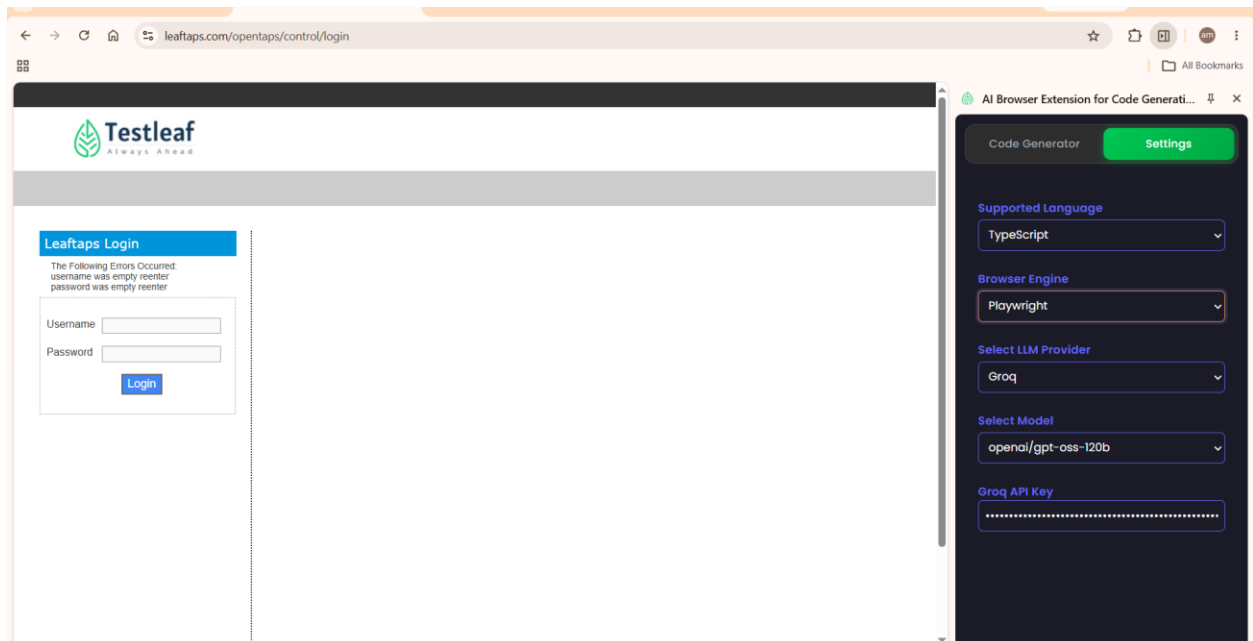
Type 'Yes' to approve



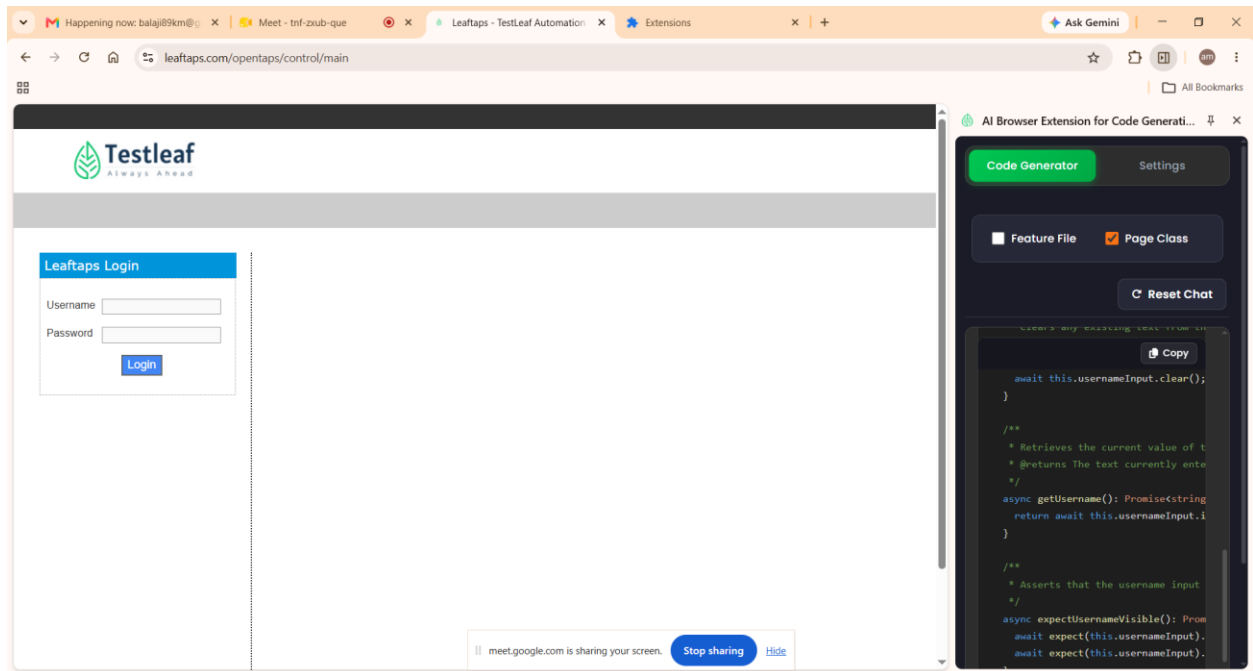
Click on Keep



Open Extension browser newly and select the fields as shown in the below screenshot



Click on Code Generator. Select Page Class. Click on 'Inspect'. Click on Username field. Click on stop. Click on Generate.



```
import { Locator, Page, expect } from '@playwright/test';
```

```
/**
```

```
 * Page Object representing a login screen containing the username input.
```

```
 */
```

```
export class LoginPage {
```

```
  readonly page: Page;
```

```
  readonly usernameInput: Locator;
```

```
  /**
```

```
   * @param page Playwright Page instance.
```

```
   */
```

```
  constructor(page: Page) {
```

```
    this.page = page;
```

```
this.usernameInput = page.locator('#username');
}

/**
 * Navigates to the login page.
 * @param url Relative or absolute URL to open. Defaults to '/'.
 */
async open(url: string = '/'): Promise<void> {
    await this.page.goto(url);
}

/**
 * Fills the username input with the provided value.
 * @param username Text to type into the username field.
 */
async fillUsername(username: string): Promise<void> {
    await this.usernameInput.fill(username);
}

/**
 * Clears any existing text from the username input.
 */
async clearUsername(): Promise<void> {
    await this.usernameInput.clear();
}
```

```
/**  
 * Retrieves the current value of the username input.  
 * @returns The text currently entered in the username field.  
 */  
async getUsername(): Promise<string> {  
    return await this.usernameInput.inputValue();  
}
```

```
/**  
 * Asserts that the username input is visible and enabled.  
 */  
async expectUsernameVisible(): Promise<void> {  
    await expect(this.usernameInput).toBeVisible();  
    await expect(this.usernameInput).toBeEnabled();  
}  
}
```